C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory, Edge Cases & Code Examples

- ✓ Unpacking Structured Data
- ✓ Compile-Time Conditional Branching
- ✓ Header-Only Variable Definitions
- ✓ Structured Bindings and Temporaries
- ✓ if constexpr vs Runtime if with Type Traits

→ *Swipe to learn* →

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 1. Structured Bindings - Unpacking Structured Data (1/3)

Definition: Structured bindings provide syntactic sugar for unpacking structured types (pairs, tuples, structs, arrays) into individual named variables in a single declaration.

Syntax and Basic Usage:

Syntax Form	Effect	Example
`auto [a, b] = obj;`	Copy values	`auto [x, y] = pair{1, 2};`
`auto& [a, b] = obj;`	References (modifiable)	`auto& [x, y] = point;`
`const auto& [a, b] = obj;`	Const references (read-only)	`const auto& [k, v] = map_entry;`
`auto&& [a, b] = obj;`	Forwarding references	`auto&& [x, y] = std::move(pair);`

Comparison with Legacy Syntax:

```
// ❌ C++14: Verbose and error-prone
auto p = std::make_pair(1, "hello");
int first = p.first;
std::string second = p.second;

// ✅ C++17: Clean and readable
auto [num, text] = std::make_pair(1, "hello");
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 1. Structured Bindings - Unpacking Structured Data (2/3)

Supported Type Categories:

Type Category	Requirement	Example
Fixed-size arrays	Known size at compile time	`int arr[3] = {1,2,3};` `auto [a, b, c] = arr;`
std::pair	Standard library pair type	`auto [key, value] = myMap.at(id);`
std::tuple	Standard library tuple (any arity)	`auto [x, y, z] = std::make_tuple(1, 2.5, 'c');`
Aggregate structs	All public members, no user constructors	`struct Point { int x, y; };` `auto [px, py] = Point{10, 20};`

Copy vs Reference Semantics:

```
struct Point { int x = 1; int y = 2; };
Point p;

// Copy semantics - modifications don't affect original
auto [a, b] = p;
a = 10;
std::cout << p.x;  // Still 1

// Reference semantics - modifications propagate
auto& [c, d] = p;
c = 20;
std::cout << p.x;  // Now 20
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 1. Structured Bindings - Unpacking Structured Data (3/3)

Common Use Cases:

- **Map iteration:** `for (const auto& [key, value] : sensorMap)` - eliminates `.first` / `.second`
- **Function returns:** `auto [status, data] = parseMessage();` - clean multi-value returns
- **Sensor data:** `auto [temp, pressure, humidity] = readEnvironment();` - descriptive names for tuple elements
- **Error handling:** `auto [success, errorCode] = validateInput();` - explicit status tracking

Key Limitations:

Limitation	Reason	Workaround
Cannot capture in lambda directly	Bindings are hidden names	Use `[=]` or `[&]` to capture all
Exact count match required	Compile-time safety	Ensure binding count matches type size
No temporary lifetime extension	Standard limitation	Bind to named object, not temporary
No private member support	Requires aggregate type	Make members public or use getter returning tuple

---

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 2. if constexpr - Compile-Time Conditional Branching (1/3)

Definition: `if constexpr` evaluates conditions at compile time and **completely discards** non-taken branches before code generation, enabling type-dependent code in templates without complex metaprogramming.

Core Principle:

Regular `if`: Compiles both branches, evaluates condition at runtime

`if constexpr`: Evaluates condition at compile time, discards dead branch entirely (allows invalid code in unused branch)

Comparative Analysis:

Feature	`if`	`if constexpr`
Evaluation time	Runtime	Compile time
Condition requirement	Any boolean expression	Constant expression
Dead branch compilation	Must be syntactically valid	Can be invalid (discarded)
Template instantiation	Both branches instantiated	Only taken branch instantiated
Performance overhead	Runtime branch check	Zero overhead (eliminated)
Primary use case	Runtime decision-making	Template metaprogramming
Binary size impact	Both branches in code	Only taken branch in code

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 2. if constexpr - Compile-Time Conditional Branching (2/3)

Practical Example - Why This Matters:

```
template<typename T>
void problematic(T val) {
    if (std::is_integral_v<T>) {  // ❌ Regular if - both branches compiled
        std::cout << val + 1;
    } else {
        std::cout << val.size();  // ❌ ERROR: int has no .size() method!
    }
}
// Even though condition is compile-time constant, regular if requires both branches valid

template<typename T>
void correct(T val) {
    if constexpr (std::is_integral_v<T>) {  // ✅ Compile-time if
        std::cout << val + 1;
    } else {
        std::cout << val.size();  // ✅ OK: not compiled when T=int
    }
}
```

Replacing Complex Metaprogramming Patterns:

C++14 Pattern	C++17 with if constexpr	Readability Gain
SFINAE with enable_if	Single function with if constexpr branches	High - eliminates cryptic enable_if
Tag dispatching	Direct type trait checks in if constexpr	High - no helper tag types needed
Template specializations	Single template with conditional branches	Medium - fewer overloads to maintain
Recursive template termination	Base case in if constexpr	High - clear termination condition

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 2. if constexpr - Compile-Time Conditional Branching (3/3)

Before/After Comparison - SFINAE Elimination:

```
// ❌ C++14: SFINAE approach (verbose, hard to debug)
template<typename T>
std::enable_if_t<std::is_integral_v<T>, void>
process(T val) {
    std::cout << val + 1;
}

template<typename T>
std::enable_if_t<!std::is_integral_v<T>, void>
process(T val) {
    std::cout << val;
}

// ✅ C++17: if constexpr (clean, readable)
template<typename T>
void process(T val) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << val + 1;
    } else {
        std::cout << val;
    }
}
```

Common Use Patterns:

- **Type-based algorithm selection:** Different implementations for arithmetic vs container types
- **Variadic template recursion:** Clean base case handling without separate specialization
- **Compile-time optimization:** Eliminate debug code paths in release builds
- **Type trait branching:** Direct use of `std::is_*_v` traits for conditional logic

---

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 3. Inline Variables - Header-Only Variable Definitions (1/4)

The Problem (Pre-C++17):

Before C++17, the **One Definition Rule (ODR)** prohibited defining static variables in headers. Each translation unit including the header would create a separate copy, causing linker errors or multiple instances.

ODR Violation Example:

```
// ❌ C++14: header.h
struct Config {
    static constexpr int MaxSize = 100;  // Declaration only
};
// Problem: Each TU creates its own copy - ODR violation!

// Required: config.cpp
constexpr int Config::MaxSize;  // Definition in .cpp file
```

The C++17 Solution:

The `inline` keyword tells the linker to **merge multiple definitions** across translation units into a single variable, similar to inline functions.

```
// ✅ C++17: header.h (complete, no .cpp needed)
struct Config {
    inline static constexpr int MaxSize = 100;  // Declaration AND definition
};
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 3. Inline Variables - Header-Only Variable Definitions (2/4)

Inline Variable Semantics:

Aspect	Behavior	Implication
Linkage	Multiple definitions merged by linker	Same variable across all TUs
Initialization	Happens once (implementation-defined which TU)	Consistent state guaranteed
Mutability	Can be non-const (use carefully!)	Enables global mutable state
Constexpr compatibility	Works with constexpr (compile-time init)	Best practice for constants

Use Case Comparison:

Scenario	C++14 Approach	C++17 with inline
Static class member	Declaration in header + definition in .cpp	`inline static` in header only
Global constant	`extern` in header + definition in .cpp	`inline constexpr` in header only
Template static member	Definition in header (risky, ODR-prone)	`inline static` (safe, ODR-compliant)
Mutable global config	Definition in single .cpp file only	`inline` in header (shared across TUs)

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 3. Inline Variables - Header-Only Variable Definitions (3/4)

Header-Only Library Pattern:

```
// config.h - Complete header-only configuration
#ifndef CONFIG_H
#define CONFIG_H

namespace AppConfig {
    // ✅ Constants - compile-time initialization
    inline constexpr int MaxConnections = 100;
    inline constexpr double Timeout = 30.0;

    // ✅ Non-const runtime state - use sparingly!
    inline int ActiveConnections = 0;

    // ✅ Template static members
    template<typename T>
    struct Limits {
        inline static constexpr T max_value = T(1000);
    };
}

#endif

// No .cpp file needed - everything in header!
```

Inline vs Constexpr Clarification:

Keyword	Meaning	When to Use
`constexpr`	Value computed at compile time, implicitly const	Compile-time constants
`inline`	Merge multiple definitions across TUs	Header variables (mutable or const)
`inline constexpr`	Both properties combined	**Best practice for header constants**

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Theory

📌 3. Inline Variables - Header-Only Variable Definitions (4/4)

Best Practices:

- **Prefer** `inline constexpr` for constants in headers (compile-time + ODR-safe)
- **Avoid mutable inline variables** - creates global state (hard to test/reason about)
- **Use for configuration values** - single source of truth across modules
- **Enable header-only libraries** - simplifies distribution and usage

Important Caveats:

- **Static initialization order fiasco:** Inline doesn't solve initialization order dependencies between TUs
- **Must be consistent:** All TUs must see identical initialization expressions
- **Cannot forward-declare:** `extern inline` is invalid syntax
- **Linkage requirements:** Only needed for namespace-scope or static class members (not local variables)

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Edge Cases

📌 1. Structured Bindings and Temporaries

```
// ❌ Dangling reference
const auto& [x, y] = std::make_pair(1, 2);  // Temporary destroyed!
std::cout << x;  // UB - x refers to destroyed temporary

// ✅ Correct - extends lifetime
const auto [x, y] = std::make_pair(1, 2);  // Copy, not reference
std::cout << x;  // OK

// ✅ Also correct - bind to named variable
auto p = std::make_pair(1, 2);
const auto& [x, y] = p;  // Reference to p, which is still alive
std::cout << x;  // OK
```

When structured bindings use references (`auto&` or `const auto&`), they reference the original object. If that object is a temporary, the reference dangles once the full expression ends. This is different from `const auto&` extending temporary lifetime for regular variables.

Key takeaway: Structured bindings don't extend temporary lifetimes; always bind to a named object when using references.

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Edge Cases

📌 2. if constexpr vs Runtime if with Type Traits

```
template<typename T>
void problematic(T val) {
    if (std::is_integral_v<T>) {  // ❌ Runtime if
        std::cout << val + 1;  // OK for integers
    } else {
        std::cout << val.size();  // ❌ Compilation error for integers!
    }
}
// Even though the condition is compile-time constant, regular if compiles BOTH branches
// The else branch will fail for T=int because int has no .size()

template<typename T>
void correct(T val) {
    if constexpr (std::is_integral_v<T>) {  // ✅ Compile-time if
        std::cout << val + 1;
    } else {
        std::cout << val.size();  // Not compiled when T=int
    }
}
```

With regular `if`, the compiler must ensure both branches are syntactically valid for the instantiated type, even if the condition is known at compile time. `if constexpr` discards the non-taken branch before syntax checking, allowing type-specific code.

Key takeaway: Use `if constexpr` whenever you have type-dependent code that wouldn't compile for all template instantiations.

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Edge Cases

📌 3. Inline Variables and ODR

```
// header.h
inline int global_counter = 0;  // ✅ OK - merged across TUs

// a.cpp
#include "header.h"
void increment_a() { ++global_counter; }

// b.cpp
#include "header.h"
void increment_b() { ++global_counter; }

// Both functions modify THE SAME global_counter (not separate copies)
```

Without `inline`, each translation unit would create its own `global_counter`, violating ODR and causing linker errors. With `inline`, the linker merges all definitions into a single variable.

Important: Non-const inline variables can be modified, making them true global mutable state. Use carefully!

Key takeaway: Inline variables enable header-only mutable global state, but use sparingly to avoid coupling and testing difficulties.

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Edge Cases

📌 4. Structured Bindings Cannot Be Captured in Lambdas

```
auto [x, y] = std::make_pair(1, 2);

// ❌ Invalid - can't capture structured bindings directly
auto bad = [x, y]() { return x + y; };

// ✅ Workaround 1 - capture by value (copies)
auto good1 = [=]() { return x + y; };

// ✅ Workaround 2 - capture by reference
auto good2 = [&]() { return x + y; };

// ✅ Workaround 3 - capture the original pair
auto p = std::make_pair(1, 2);
auto good3 = [p]() { return p.first + p.second; };
```

Structured bindings create hidden variables that cannot be named in lambda capture lists. You must use `[=]`, `[&]`, or capture the original object.

Key takeaway: Structured bindings are syntactic sugar that introduces hidden names; capture the original object or use `[=]` / `[&]` for lambdas.

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Edge Cases

📌 5. if constexpr in Non-Template Code

```
void runtime_example() {
    constexpr bool debug = true;

    if constexpr (debug) {  // Still a compile-time branch
        std::cout << "Debug mode\n";
    }
}
```

`if constexpr` works in non-template code too, but it's less useful because the condition must still be a constant expression. The main benefit is eliminating dead code from the binary.

However, regular `if` with a constexpr condition will often be optimized away by the compiler anyway, so `if constexpr` in non-template code is rare.

Key takeaway: `if constexpr` is primarily for templates; in non-template code, the compiler often optimizes constexpr conditions in regular `if` statements anyway.

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 1. Structured Bindings with Maps

Structured bindings make map iteration dramatically more readable. The `[id, reading]` syntax clearly names both the key and value, whereas `entry.first` and `entry.second` require mental translation.

In autonomous driving systems, this pattern is common for processing multiple sensor streams where each sensor has an ID and data payload.

```
#include <iostream>
#include <map>
#include <string>

void process_sensor_data() {
    std::map<int, std::string> sensors = {
        {1, "Temperature: 25C"},
        {2, "Pressure: 101kPa"},
        {3, "Humidity: 60%"}
    };

    // ✅ Clean iteration with structured bindings
    for (const auto& [id, reading] : sensors) {
        std::cout << "Sensor " << id << ": " << reading << "\n";
    }

    // Compare with C++11/14 approach
    for (const auto& entry : sensors) {
        std::cout << "Sensor " << entry.first << ": " << entry.second << "\n";
    }
}
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 2. if constexpr for Type-Dependent Algorithms (1/2)

This demonstrates how `if constexpr` enables writing a single generic function that specializes behavior based on type characteristics, without needing separate overloads or template specializations.

```
#include <iostream>
#include <type_traits>
#include <vector>

template<typename Container>
void print_size(const Container& c) {
    if constexpr (std::is_array_v<Container>) {
        std::cout << "Array size: " << std::extent_v<Container> << "\n";
    } else if constexpr (requires { c.size(); }) {  // C++20 concept
        std::cout << "Container size: " << c.size() << "\n";
    } else {
        std::cout << "Size unknown\n";
    }
}

// Alternative using type traits for C++17
template<typename T>
void process_data(T data) {
    if constexpr (std::is_integral_v<T>) {
        // Integer-specific processing
        std::cout << "Processing integer: " << data * 2 << "\n";
    } else if constexpr (std::is_floating_point_v<T>) {
        // Floating-point specific processing
        std::cout << "Processing float: " << data / 2.0 << "\n";
    } else {
        // Generic processing
        std::cout << "Processing other type\n";
    }
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    std::vector<int> vec = {1, 2, 3};

    print_size(arr);  // Array size: 5
// ...continues on next slide
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 2. if constexpr for Type-Dependent Algorithms (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    print_size(vec);  // Container size: 3

    process_data(10);      // Processing integer: 20
    process_data(3.14);    // Processing float: 1.57
    process_data("text");  // Processing other type
}
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 3. Inline Variables for Configuration

Before C++17, you'd need a config.cpp file to define these variables, breaking the header-only pattern. Now, a single header suffices, making library distribution simpler.

```
// config.h
#ifndef CONFIG_H
#define CONFIG_H

#include <string>

namespace Config {
    // ✅ C++17: Define in header without .cpp file
    inline constexpr int MaxConnections = 100;
    inline constexpr double Timeout = 30.0;
    inline const std::string ServerName = "AutoDrive-Central";

    // Template with inline static
    template<typename T>
    struct Limits {
        inline static constexpr T max_value = T(1000);
    };
}

#endif

// main.cpp
#include "config.h"
#include <iostream>

void connect_to_server() {
    std::cout << "Connecting to " << Config::ServerName
              << " (max " << Config::MaxConnections << " connections)\n";
    std::cout << "Timeout: " << Config::Timeout << "s\n";
    std::cout << "Int limit: " << Config::Limits<int>::max_value << "\n";
}

// other.cpp can include config.h and use the same variables
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 4. Structured Bindings with User-Defined Structs

For structs to work with structured bindings, they must be aggregates (no private members, no user-defined constructors). This is perfect for plain data structures like sensor readings, coordinates, or configuration bundles.

```
#include <iostream>
#include <string>

// Aggregate struct (no constructors, all public)
struct SensorReading {
    int id;
    double value;
    std::string unit;
};

SensorReading read_sensor(int id) {
    return {id, 23.5, "Celsius"};
}

void process_readings() {
    // ✅ Direct unpacking
    auto [id, temp, unit] = read_sensor(1);
    std::cout << "Sensor " << id << ": " << temp << unit << "\n";

    // ✅ With references to modify original
    SensorReading reading = {2, 100.0, "kPa"};
    auto& [sensor_id, pressure, pressure_unit] = reading;
    pressure *= 1.1;  // Calibration adjustment
    std::cout << "Adjusted: " << reading.value << "\n";  // 110.0

    // ✅ Const references for read-only
    const auto& [id2, val2, unit2] = reading;
    // val2 = 50;  // ❌ Error: const reference
}
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 5. if constexpr with Recursive Templates (1/2)

This shows how `if constexpr` simplifies variadic template recursion by providing clean base cases without needing separate template specializations.

```
#include <iostream>

// Variadic template sum using if constexpr
template<typename... Args>
auto sum(Args... args) {
    if constexpr (sizeof...(args) == 0) {
        return 0;  // Base case
    } else {
        return (... + args);  // Fold expression (C++17)
    }
}

// Recursive print with type-dependent formatting
template<typename T>
void print_value(const T& value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "[int: " << value << "]";
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "[float: " << value << "]";
    } else {
        std::cout << "[other: " << value << "]";
    }
}

template<typename First, typename... Rest>
void print_all(const First& first, const Rest&... rest) {
    print_value(first);
    if constexpr (sizeof...(rest) > 0) {
        std::cout << ", ";
        print_all(rest...);  // Recursive call
    }
}

int main() {
    std::cout << "Sum: " << sum(1, 2, 3, 4, 5) << "\n";  // 15
// ...continues on next slide
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 5. if constexpr with Recursive Templates (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    print_all(42, 3.14, "text", 'A');
    // Output: [int: 42], [float: 3.14], [other: text], [other: A]
}
```

C++17 Language Features - Structured Bindings, if constexpr, Inline Variables

Code Examples

📌 6. Combining All Three Features

This example demonstrates real-world usage combining all three features: structured bindings for clean map iteration, if constexpr for compile-time debug elimination, and inline variables for header-only configuration.

```
#include <iostream>
#include <map>
#include <string>
#include <type_traits>

// Inline configuration
namespace AppConfig {
    inline constexpr bool EnableDebug = true;
    inline const std::string AppName = "SensorHub";
}

// Generic logging function using if constexpr
template<typename T>
void log(const std::string& label, const T& value) {
    if constexpr (AppConfig::EnableDebug) {
        std::cout << "[" << AppConfig::AppName << "] " << label << ": ";

        if constexpr (std::is_arithmetic_v<T>) {
            std::cout << value << " (numeric)\n";
        } else {
            std::cout << value << " (text)\n";
        }
    }
    // If EnableDebug is false, entire function body is eliminated!
}

void process_sensors() {
    std::map<int, double> sensor_data = {{1, 25.3}, {2, 30.7}};

    // Structured bindings in loop
    for (const auto& [id, temp] : sensor_data) {
        log("Temperature", temp);
        log("SensorID", id);
    }
}
```

What You Learned

Key Takeaways

- ✓ Unpacking Structured Data
- ✓ Compile-Time Conditional Branching
- ✓ Header-Only Variable Definitions
- ✓ Structured Bindings and Temporaries

→ *Swipe to learn* →